ADO Self-Hosted Agents: Shared Pool Design

Purpose

This document defines how the central **CI/CD Pipeline team** provisions and operates a shared pool of Azure DevOps self-hosted agents in a single Azure subscription for the **EPIC pipeline framework** — the sole CI/CD pipeline for the enterprise.

Because EPIC is the only pipeline, the design is deliberately simple: one pool, one image, one identity, one Key Vault. Every build flows through the EPIC orchestrator and engine, so every guarantee (auth, tagging, networking, tooling) is enforced in one place with 100% coverage. There is nothing for app teams to configure beyond their `epic.json`.

Design Principles

- **Zero auth burden on app teams** — agents authenticate to all tools via a single Managed Identity; secrets live in one Key Vault; the EPIC engine injects them automatically
- **Guaranteed chargeback** — the EPIC engine requires a `chargeback` section in `epic.json` and tags every build; builds without it fail fast — 100% cost attribution by default
- **Corporate network connectivity** — agents run inside a VNET peered to PG&E's network, with direct access to SonarQube, Wiz, JFrog, and other internal tools
- **Fully managed** — Microsoft manages the VMs (Managed DevOps Pools); the CI/CD Pipeline team manages the config; app teams just write code
- **Single pipeline, single path** — EPIC controls every stage (build, test, scan, deploy), so the CI/CD Pipeline team knows exactly what the agents need and can guarantee consistency across every build in the enterprise

Ownership Model

Responsibility	Owner
Pool, scaling, images, EPIC engine/templates	CI/CD Pipeline team
VNET, NSGs, peering to PG&E network	CI/CD Pipeline team + Network team
Key Vault secrets (tool credentials)	CI/CD Pipeline team
Azure resource tags, cost reporting	CI/CD Pipeline team + FinOps
`epic.json` (app config + chargeback fields)	Application teams (one-time setup)

---

1. Compute: Managed DevOps Pools

Managed DevOps Pools (MDP) is the sole compute strategy. Microsoft owns and manages the underlying VMs; the CI/CD Pipeline team configures the pool through Azure portal or Bicep/Terraform.

Why MDP: - No VM lifecycle to manage (no image generalization, no manual scale set updates) - Scales in increments of 1 agent (cost-efficient, no over-provisioning) - Flexible standby scheduling (business hours vs. off-hours) - Supports thousands of agents per pool - Supports multiple images per pool via aliases (future-proofs for Windows if needed) - Managed Identity integration for Key Vault access

The CI/CD Pipeline team creates and owns the MDP resource. App teams never interact with Azure infrastructure — they don't even choose a pool. The EPIC orchestrator hardcodes the pool reference, so every build runs on `epic-agents` automatically.

---

2. Authentication: Managed Identity + Key Vault

This is the core of the "any agent, zero auth management" goal. Every agent automatically has access to every tool credential without app teams doing anything.

How It Works

```
MDP Pool
  └── Managed Identity (system-assigned)
        └── Authorized to read secrets from:
              └── Azure Key Vault ("kv-epic-cicd")
                    ├── sonarqube-token
                    ├── wiz-client-id
                    ├── wiz-client-secret
                    ├── jfrog-api-key
                    ├── github-pat
                    └── (any future tool credentials)
```

1. The MDP pool has a **system-assigned Managed Identity**
2. That identity has `Get` and `List` permissions on a single **Azure Key Vault** (`kv-epic-cicd`)
3. **ADO variable groups** are linked to this Key Vault — each variable maps to a secret name
4. The EPIC engine references this variable group once — since EPIC is the only pipeline, every build in the enterprise automatically gets the credentials at runtime
5. When a credential rotates, the CI/CD Pipeline team updates it in Key Vault once — every build picks it up on the next run with zero changes

ADO Variable Group Setup

```
Variable Group: "EPIC-Tool-Credentials"
  Source: Azure Key Vault (kv-epic-cicd)
  Variables:
    - SONARQUBE_TOKEN    → sonarqube-token
    - WIZ_CLIENT_ID      → wiz-client-id
    - WIZ_CLIENT_SECRET  → wiz-client-secret
    - JFROG_API_KEY      → jfrog-api-key
    - GITHUB_PAT         → github-pat
```

This variable group is linked once in `epic-engine.yml`. Since EPIC is the only pipeline, this single linkage covers every build in the enterprise.

What App Teams Experience

Nothing. They define their `epic.json` and their builds authenticate to SonarQube/Wiz/JFrog automatically. There is no way for a team to run a build outside of EPIC, so there is no way to accidentally skip auth setup. If a new tool is added, the CI/CD Pipeline team adds the secret to Key Vault, adds it to the variable group, and updates the relevant EPIC stage template. No app-side changes, no notifications needed.

---

3. Networking: VNET with PG&E Peering

Agents must reach PG&E internal tools (SonarQube, Wiz, JFrog Artifactory) and external services (ADO, package registries).

Network Topology

```
PG&E Corporate Network
  ├── SonarQube (sonarqube.pge.com)
  ├── Wiz (wiz-agent endpoint)
  ├── JFrog Artifactory (artifactory.pge.com)
  │
  └── VNET Peering / ExpressRoute
        │
        └── Azure VNET: "vnet-epic-cicd"
              └── Subnet: "snet-ado-agents"
                    └── MDP Pool (VNET-injected)
```

Configuration

- MDP uses the **"bring your own VNET"** option to place agents in `snet-ado-agents`
- VNET is peered to PG&E's corporate network (or connected via ExpressRoute, depending on existing infrastructure)
- **NSG on the agent subnet** allows:

    - **Outbound to PG&E network**: SonarQube, Wiz, JFrog endpoints (specific IPs or CIDR ranges)
    - **Outbound to Azure/Internet**: `dev.azure.com`, `*.visualstudio.com`, `vstsagenttools.blob.core.windows.net`, package feeds (NuGet, npm, PyPI)
    - **No inbound from internet** — agents only initiate outbound connections
- **Private Endpoints** for Azure services used by pipelines (Key Vault, ACR, Storage)

The CI/CD Pipeline team owns the VNET and NSG rules. App teams don't configure networking.

---

4. Pool Topology

One pool. Since EPIC is the only pipeline and controls every stage, there's no need for multiple pools, demands, or agent capabilities routing. The EPIC orchestrator hardcodes the pool:

```yaml
# epic-orchestrator.yml
pool:
  name: epic-agents
```

The Pool

Pool	Image	Workloads
`epic-agents`	Ubuntu 24.04 + .NET 10, Node, Terraform, AWS CLI, Azure CLI, Docker, SonarScanner, Wiz CLI, JFrog CLI	Everything

The image contains every tool that any EPIC stage might invoke. This is manageable because the CI/CD Pipeline team owns both the image and the stage templates — they know exactly what's needed and nothing extraneous gets installed.

If Windows builds become necessary in the future, MDP supports multiple images per pool with aliases. Add a Windows image to the same pool and dispatch via the EPIC engine based on `appType` — still one pool.

Scaling

Setting	Value	Rationale
Maximum agents	20 (adjust based on demand)	Cost ceiling

Setting	Value	Rationale
Standby schedule	Weekday 08:00–18:00: 4 standby; Off-hours: 0	Matches developer working hours
Agent mode	Stateless (fresh VM per job)	Clean builds, no state leakage between teams
Grace period	15 minutes	Absorbs bursts without over-scaling

---

5. Chargeback: `epic.json` + Automatic Build Tags

Because EPIC is the only pipeline, chargeback has **100% coverage by design**. The EPIC engine requires the `chargeback` section in `epic.json` — builds without it fail immediately. There is no path for a build to run untagged.

epic.json Extension

Add a **required** `chargeback` section to the existing pipeline contract:

```
{
  "app": {
    "appName": "my-app",
    "appType": "dotnet",
    "codePath": "src/"
  },
  "cloud": {
    "awsAccountId": "123456789012"
  },
  "chargeback": {
    "costCenter": "CC-1234",
    "businessUnit": "Gas-Operations"
  }
}
```

App teams fill this in once when onboarding to EPIC. The EPIC engine validates at runtime that both `costCenter` and `businessUnit` are present and non-empty — if missing, the pipeline fails before any stages run, with a clear error message pointing the team to the field.

Automatic Tagging in the EPIC Engine

The EPIC engine reads the `chargeback` section and tags every build. This runs as the first step in `epic-engine.yml`, before download/build/test/scan/deploy stages:

```
# epic-engine.yml — first step, before any stages
steps:
```

```
  - script: |
      if [ -z "$(chargeback.costCenter)" ] || [ -z "$(chargeback.businessUnit)
" ]; then
        echo "##vso[task.logissue type=error]epic.json is missing required ch
argeback fields (costCenter, businessUnit)"
        exit 1
      fi
      echo "##vso[build.addbuildtag]costcenter:$(chargeback.costCenter)"
      echo "##vso[build.addbuildtag]bu:$(chargeback.businessUnit)"
      echo "##vso[build.addbuildtag]project:$(System.TeamProject)"
      echo "##vso[build.addbuildtag]app:$(app.appName)"
      echo "##vso[build.addbuildtag]apptype:$(app.appType)"
    displayName: 'Validate and apply chargeback tags'
```

Every build is tagged with:

Tag	Source	Example
`costcenter:{code}`	`epic.json` chargeback.costCenter	`costcenter:CC-1234`
`bu:{unit}`	`epic.json` chargeback.businessUnit	`bu:Gas-Operations`
`project:{name}`	ADO system variable	`project:my-project`
`app:{name}`	`epic.json` app.appName	`app:my-app`
`apptype:{type}`	`epic.json` app.appType	`apptype:dotnet`

Because EPIC is the only pipeline, this tagging step is the single enforcement point for the entire enterprise. No build runs without tags.

Azure Resource Tags (on the Pool Infrastructure)

The CI/CD Pipeline team applies these tags to the MDP resource and its resource group:

Tag Key	Value
`Service`	`ADO-Agents`
`ManagedBy`	`CICD-Pipeline-Team`
`Environment`	`Shared-CI`
`CostCenter`	`CC-0000` (platform team's cost center)

Enable **Tag Inheritance** in Azure Cost Management so resource group tags flow to all child resources automatically.

Chargeback Calculation

```
Monthly Pool Cost (Azure Cost Management, filtered by Service:ADO-Agents)
  ÷ Total Build Minutes (all builds on the pool that month)
  × Team Build Minutes (builds tagged with a specific costcenter/bu)
  = Team Chargeback Amount
```

Data sources: - **Azure Cost Management** → total pool cost (exports filtered by `Service:ADO-Agents` tag) - **ADO Analytics OData** → per-build duration grouped by `costcenter:` and `bu:` tags

The CI/CD Pipeline team produces a monthly report (Power BI or Azure Workbook) and sends it to FinOps.

---

6. Golden Image Management

One image, maintained by one team. Since EPIC controls every stage, the CI/CD Pipeline team knows exactly what tools the image needs — there are no surprise requirements from ad-hoc pipelines.

Pipeline

1. **Packer** builds from a base Ubuntu 24.04 marketplace image
2. Installs every tool that any EPIC stage invokes: .NET 10 SDK, Node LTS, Terraform, AWS CLI, Azure CLI, Docker, SonarScanner, Wiz CLI, JFrog CLI
3. Does **not** install the ADO agent binary (MDP handles this)
4. Publishes to **Azure Compute Gallery**
5. Updates the MDP image reference
6. Runs on a **weekly schedule** (or on-demand for urgent patches)

Adding a New Tool

Because EPIC owns all stages, adding a tool is a single, coordinated change: 1. CI/CD Pipeline team adds the tool to the Packer template 2. Adds the credential to Key Vault and the variable group (if the tool requires auth) 3. Adds or updates the EPIC stage template that invokes the tool 4. Triggers an image rebuild 5. Every build in the enterprise gets the new tool automatically — no app-side changes, no communication needed

---

7. Security

Concern	Mitigation
Cross-team credential leakage	Stateless agents (fresh VM per job); secrets injected at runtime, never persisted

Concern	Mitigation
Credential rotation	CI/CD Pipeline team updates Key Vault; no pipeline or agent changes needed
Network exposure	Agents in a private subnet; no inbound internet; outbound restricted by NSG
Image supply chain	Images built from verified marketplace bases by the CI/CD Pipeline team
Agent privilege	MDP agents run as non-admin user by default
Key Vault access	Only the MDP Managed Identity can read secrets; no human access from pipelines

---

8. Implementation Roadmap

Phase	What	Outcome
1	Provision VNET, subnet, NSG, peering to PG&E network. Create Key Vault with tool credentials. Enable Cost Management tag inheritance.	Infrastructure ready
2	Build Packer image pipeline. Publish first golden image to Compute Gallery.	Repeatable image process
3	Create MDP pool (`epic-agents`). Configure VNET injection, scaling, Managed Identity. Link Key Vault to ADO variable group. Update `epic-orchestrator.yml` to target `epic-agents` pool.	Agents operational, EPIC wired up
4	Add required `chargeback` section to `epic.json` schema. Add validation + tagging step to EPIC engine. Pilot with 2-3 teams.	100% chargeback coverage on pilot builds

Phase	What	Outcome
5	Build monthly chargeback report (Power BI / Azure Workbook). Deliver to FinOps.	Cost attribution live
6	Migrate remaining teams from Microsoft-hosted agents to `epic-agents`. Adjust pool sizing based on observed demand.	Full enterprise rollout, single pool

---

References

- Managed DevOps Pools Overview
- Compare MDP with VMSS Agents
- MDP Networking (VNET injection)
- MDP Identity (Managed Identity)
- Azure Cost Management Tag Inheritance
- ADO REST API: Build Tags